```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Dataset
data = """artificial intelligence systems learn patterns from data
sequence models process information step by step
recurrent neural networks are useful for sequence prediction
lstm networks handle long term dependencies
deep learning models improve sequence learning
generative models create new samples from learned patterns
language models predict the next word in a sentence
sequence generation is used in chatbots and assistants
machine learning helps computers learn automatically
training data improves model accuracy
neural networks simulate human brain structures
optimization algorithms improve learning efficiency
technology is transforming modern education
online learning platforms use artificial intelligence
students benefit from intelligent tutoring systems
automation improves productivity and decision making"""

# Tokenization
tokenizer = Tokenizer()
tokenizer.fit_on_texts([data])

total_words = len(tokenizer.word_index) + 1

# Create sequences
input_sequences = []
for line in data.split("\n"):
    token_list = tokenizer.texts_to_sequences([line])[0]
    for i in range(1, len(token_list)):
        input_sequences.append(token_list[:i+1])

# Padding
max_seq_len = max(len(x) for x in input_sequences)
input_sequences = pad_sequences(input_sequences, maxlen=max_seq_len, padding='pre')

# Split into X and y
X = input_sequences[:, :-1]
y = input_sequences[:, -1]

# One-hot encoding
y = tf.keras.utils.to_categorical(y, num_classes=total_words)

# Model
model = Sequential()
model.add(Embedding(total_words, 50, input_length=max_seq_len-1))
model.add(LSTM(100))
model.add(Dense(total_words, activation='softmax'))

model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

# Training
model.fit(X, y, epochs=100, verbose=1)

# Generate text
def generate_text(seed_text, next_words=5):
    for _ in range(next_words):
        token_list = tokenizer.texts_to_sequences([seed_text])[0]
        token_list = pad_sequences([token_list], maxlen=max_seq_len-1, padding='pre')

        predicted = np.argmax(model.predict(token_list), axis=-1)[0]

        for word, index in tokenizer.word_index.items():
            if index == predicted:
                seed_text += " " + word
                break
    return seed_text

# Example Output
print(generate_text("machine learning", 5))
```

```
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.7955 - loss: 1.2020
Epoch 78/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.8409 - loss: 1.1771
Epoch 79/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 0.8295 - loss: 1.1558
Epoch 80/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 0.8523 - loss: 1.1391
Epoch 81/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8750 - loss: 1.1088
Epoch 82/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.8636 - loss: 1.0887
Epoch 83/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.8750 - loss: 1.0660
Epoch 84/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8864 - loss: 1.0524
Epoch 85/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.8864 - loss: 1.0261
Epoch 86/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.8977 - loss: 1.0039
Epoch 87/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.8977 - loss: 0.9797
Epoch 88/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8977 - loss: 0.9690
Epoch 89/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9318 - loss: 0.9446
Epoch 90/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9091 - loss: 0.9255
Epoch 91/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.8977 - loss: 0.9116
Epoch 92/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8977 - loss: 0.8996
Epoch 93/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9091 - loss: 0.8685
Epoch 94/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9205 - loss: 0.8606
Epoch 95/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8977 - loss: 0.8457
Epoch 96/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9091 - loss: 0.8210
Epoch 97/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9205 - loss: 0.8103
Epoch 98/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9205 - loss: 0.7944
Epoch 99/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9205 - loss: 0.7811
Epoch 100/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9318 - loss: 0.7622
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 219ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
machine learning helps computers learn automatically automatically
```